Institut Teknologi Bandung

Directorate of Continuing Professional Education

in partnership with

CHAPTER 10

Interpreting What ConvNets Learn

Four ways to open the box — and the argument that for ConvNets, the box was never as closed as people say.

Duration 2.5 hours

Instructor Prof. Bambang Riyanto Trilaksono

Source Chollet & Watson, Deep Learning with Python 3e — chapter 10

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. Build a **multi-output model** that returns intermediate activations, and read what each depth of layer is doing.
2. Explain **information distillation** and the rising sparsity of activations with depth.
3. Visualise what a filter responds to using **gradient ascent in input space**, and say how it differs from ordinary training.
4. Produce a **Grad-CAM heatmap** and use it to explain — or debug — a particular classification.
5. Project a model's **latent space** to two dimensions to find outliers and semantically ambiguous samples.
6. Say which technique answers which question, so the right one is reached for in a real investigation.

BOOK
Chapter 10 — full text

NOTEBOOK
01_intermediate_activations.ipynb

NOTEBOOK
02_filter_visualisation.ipynb

NOTEBOOK
03_grad_cam.ipynb

NOTEBOOK
04_latent_space.ipynb

REPO
Author's official notebook

The black-box claim, examined

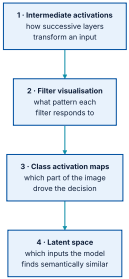
> Partially true for **certain types** of model. **Definitely not true for ConvNets** — their representations are highly amenable to visualisation, largely because they are **representations of visual concepts**

Since 2013 a wide array of techniques has been developed. This chapter covers the four most accessible and useful.

Why this matters beyond curiosity

This is especially relevant where deep learning **complements human expertise** rather than replacing it — medical imaging being the example the book names. **An unexplained disagreement is not usable in that setting.**

The four techniques, and what each answers



Roughly in order of effort. The first uses your own small ConvNet; the next two use a pretrained Xception.

01

Visualising intermediate activations

What each layer does to one particular image.

SECTION 10.1

What we are looking at

Feature maps have three dimensions — width, height, and channels. Because **each channel encodes a relatively independent feature**, the right way to look at them is **one channel at a time, as a 2D image**

A model whose outputs are the layers themselves

listing 10.3 — the activation model PYTHON

```python
import keras

model = keras.models.load_model("convnet_from_scratch_with_augmentation.keras")

layer_outputs, layer_names = [], []
for layer in model.layers:
    if isinstance(layer, (keras.layers.Conv2D, keras.layers.MaxPooling2D)):
        layer_outputs.append(layer.output)
        layer_names.append(layer.name)

activation_model = keras.Model(inputs=model.input, outputs=layer_outputs)
```

This works **only because the model is Functional**. A subclassed model has no graph to reach into — exactly the trade-off chapter 7 described.

Running it, and plotting one channel

listing 10.4–10.5 — activations PYTHON

```python
activations = activation_model.predict(img_tensor)   # a list of nine arrays

first_layer_activation = activations[0]
print(first_layer_activation.shape)

import matplotlib.pyplot as plt
plt.matshow(first_layer_activation[0, :, :, 5], cmap="viridis")   # the sixth channel
```

OUTPUT

```
(1, 178, 178, 32)
```

That channel turns out to be a **diagonal edge detector** — though your own channels will differ, since the specific filters a layer learns are **not deterministic**

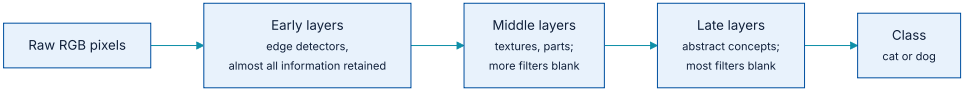
SECTION 10.1

Three things visible in the full grid

1. **The first layer is a collection of edge detectors.** At that stage the activations retain almost all the information in the original picture.
2. **Higher layers become increasingly abstract** and less visually interpretable — encoding concepts like *cat ear* and *cat eye*. They carry less about the image's appearance and more about its class.
3. **Sparsity increases with depth.** In the first layer every filter fires; further up, more and more are blank — meaning **that filter's pattern is not present in this image**.

SECTION 10.1

A deep network is an information distillation pipeline



Irrelevant information is filtered out; useful information is magnified and refined.

This is **a universal characteristic** of the representations deep networks learn, not something particular to this model.

SECTION 10.1

The bicycle test

> Try drawing a generic bicycle from memory right now. You have seen thousands, and you will almost certainly **not get it remotely right**. The effect is real, and it is the same distillation: your brain abstracted the input and discarded the detail.

Stitching every channel into one grid

listing 10.6 — the display loop PYTHON

```python
images_per_row = 16

for layer_name, layer_activation in zip(layer_names, activations):
    n_features = layer_activation.shape[-1]
    size = layer_activation.shape[1]
    n_cols = n_features // images_per_row
    display_grid = np.zeros(((size + 1) * n_cols - 1,
                             images_per_row * (size + 1) - 1))
    for col in range(n_cols):
        for row in range(images_per_row):
            channel_image = layer_activation[0, :, :, col * images_per_row + row].copy()
            # standardise so faint channels are still visible
            if channel_image.sum() != 0:
                channel_image -= channel_image.mean()
                channel_image /= channel_image.std()
                channel_image *= 64
                channel_image += 128
            channel_image = np.clip(channel_image, 0, 255).astype("uint8")
            display_grid[col * (size + 1): (col + 1) * size + col,
                         row * (size + 1): (row + 1) * size + row] = channel_image
```

One canvas per layer, sixteen channels to a row. The next slide explains the standardisation in the middle of that loop, which is the part that makes the result readable.

Why each channel is standardised first

> The per-channel standardisation matters: without it the faint channels are invisible next to the strong ones, and **the rising sparsity is exactly what you came to see**

02

Visualising ConvNet filters

Gradient ascent in input space.

SECTION 10.2

The idea: train the image instead of the weights

Same machinery, opposite target — and the sign is flipped.

Start from a blank (noisy) input and apply gradient descent to the **image**, maximising the response of one chosen filter. The result is the input that filter likes best.

Setting up: a pretrained model, and its layer names

listing 10.7–10.8 — model and layer names PYTHON

```python
import keras_hub

model = keras_hub.models.Backbone.from_preset("xception_41_imagenet")
preprocessor = keras_hub.layers.ImageConverter.from_preset(
    "xception_41_imagenet", image_size=(180, 180))

for layer in model.layers:
    if isinstance(layer, (keras.layers.Conv2D, keras.layers.SeparableConv2D)):
        print(layer.name)
```

OUTPUT

```
block2_sepconv1
block3_sepconv1
block4_sepconv1
...
block14_sepconv2
```

The names follow Xception's block structure from chapter 9, which makes it easy to sample **shallow, middle, and deep** layers for comparison.

SECTION 10.2

The loss: one filter’s mean activation

the loss to maximise

PYTHON

```python
layer_name = "block3_sepconv1"
layer = model.get_layer(name=layer_name)
feature_extractor = keras.Model(inputs=model.input, outputs=layer.output)

def compute_loss(image, filter_index):
    activation = feature_extractor(image)
    filter_activation = activation[:, 2:-2, 2:-2, filter_index]   # trim the border
    return ops.mean(filter_activation)
```

Note what this loss is a function of: **the image**. That is the whole trick.

One ascent step, in TensorFlow

listing 10.11 — gradient ascent step PYTHON

```python
@tf.function
def gradient_ascent_step(image, filter_index, learning_rate):
    with tf.GradientTape() as tape:
        tape.watch(image)                    # the image is not a Variable, so watch it
        loss = compute_loss(image, filter_index)
    grads = tape.gradient(loss, image)       # gradient w.r.t. the IMAGE
    grads = ops.normalize(grads)             # the "gradient normalization trick"
    image += learning_rate * grads           # PLUS: we are ascending, not descending
    return image
```

`tape.watch(image)` is required because only `Variable`s are watched automatically — exactly the rule chapter 3 gave. And the `+=` is what makes it **ascent rather than descent**

The same step in PyTorch and JAX

BACKEND	HOW THE GRADIENT IS OBTAINED
TensorFlow	`tape.watch(image)`, then `tape.gradient(loss, image)`.
PyTorch	`loss.backward()`, then read `image.grad`.
JAX	Transform the loss function with `jax.grad`, differentiating with respect to the image argument.

Whichever backend you use, **the image is the thing being differentiated**, and that is the only unusual part.

The loop that produces one filter’s pattern

listing 10.14 — generating a pattern PYTHON

```python
img_width = img_height = 200

def generate_filter_pattern(filter_index):
    iterations = 30
    learning_rate = 10.0
    image = keras.random.uniform(
        minval=0.4, maxval=0.6, shape=(1, img_width, img_height, 3))
    for i in range(iterations):
        image = gradient_ascent_step(image, filter_index, learning_rate)
    return image[0]
```

A learning rate of **10.0** would be absurd for training weights. Here it is fine, because **we are not trying to converge to a minimum** — only to move the image somewhere the filter likes.

…and turning the result back into an image

listing 10.15 — deprocess_image PYTHON

```python
def deprocess_image(image):
    image -= ops.mean(image)
    image /= ops.std(image)
    image *= 64
    image += 128
    image = ops.clip(image, 0, 255).astype("uint8")
    return image[25:-25, 25:-25, :]     # crop the border artefacts
```

The final crop removes the edge artefacts that gradient ascent tends to accumulate at the borders.

SECTION 10.2.4

What the filter banks look like, layer by layer

DEPTH	EXAMPLE LAYER	WHAT THE FILTERS ENCODE
Early	`block2_sepconv1`	Simple **directional edges and colours** — sometimes coloured edges.
Middle	`block4_sepconv1`	Simple **textures** made from combinations of edges and colours.
Deep	`block8_sepconv1`	Textures found in natural images: **feathers, eyes, leaves**.

> Chollet's analogy: each layer learns a **filter bank** such that its inputs can be expressed as a combination of those filters — **similar to how a Fourier transform decomposes a signal onto cosines**

The PyTorch version, written out

listing 10.12 — ascent step in PyTorch PYTHON

```python
import torch

def gradient_ascent_step(image, filter_index, learning_rate):
    image = image.clone().detach().requires_grad_(True)   # the image needs a gradient
    loss = compute_loss(image, filter_index)
    loss.backward()
    grads = image.grad
    grads = ops.normalize(grads)
    return image + learning_rate * grads
```

`requires_grad_(True)` on an **input tensor** is the PyTorch equivalent of `tape.watch(image)` — both say *differentiate with respect to this, even though it is not a parameter*.

…and the JAX version

listing 10.13 — ascent step in JAX PYTHON

```python
import jax

grad_fn = jax.grad(compute_loss)          # differentiates w.r.t. its first argument

@jax.jit
def gradient_ascent_step(image, filter_index, learning_rate):
    grads = grad_fn(image, filter_index)
    grads = ops.normalize(grads)
    return image + learning_rate * grads
```

Because `compute_loss(image, filter_index)` takes the image first, `jax.grad` differentiates with respect to it **without any extra ceremony at all**

03

Class activation heatmaps

Which part of this picture made the model say that?

SECTION 10.3

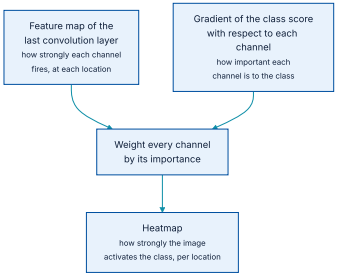
What a CAM is, and what it is for

Debugging a decision
Particularly a **misclassification** — the field is called model interpretability.

Locating an object
The heatmap tells you **where** in the frame the evidence was.

For a dogs-vs-cats model you could generate one heatmap for *cat* and another for *dog* over the same picture.

Grad-CAM, in one sentence



Weight *how strongly each channel fires here* by *how much this channel matters to the class*.

The result is a spatial map of **how intensely the input image activates the class**. The implementation follows Selvaraju et al.

The worked example: two African elephants

listing 10.17 — preparing the image PYTHON

```python
img_path = keras.utils.get_file(
    fname="elephant.jpg",
    origin="https://img-datasets.s3.amazonaws.com/elephant.jpg",
)

img = keras.utils.load_img(img_path)           # a PIL image
img_array = np.expand_dims(img, axis=0)        # to NumPy, plus a batch axis
```

Everything that follows asks the same question of this picture: **which pixels made it say *African elephant*?**

Getting the gradient of the top class

the TensorFlow version

PYTHON

```
with tf.GradientTape() as tape:
    last_conv_layer_output = last_conv_layer_model(preprocessed_image)
    tape.watch(last_conv_layer_output)
    preds = classifier_model(last_conv_layer_output)
    top_pred_index = ops.argmax(preds[0])
    top_class_channel = preds[:, top_pred_index]

grads = tape.gradient(top_class_channel, last_conv_layer_output)
```

PyTorch and JAX versions differ only in the gradient idiom, exactly as in the filter visualisation.

Pooling and weighting to get the heatmap

listing 10.23 — building the heatmap PYTHON

```python
# one number per channel: how important that channel is to the top class
pooled_grads = np.mean(grads, axis=(0, 1, 2))

last_conv_layer_output = last_conv_layer_output[0].copy()
for i in range(pooled_grads.shape[-1]):
    last_conv_layer_output[:, :, i] *= pooled_grads[i]      # weight each channel

heatmap = np.mean(last_conv_layer_output, axis=-1)          # average across channels
```

Three lines of arithmetic and no new machinery — Grad-CAM is **a weighted average, not a new model**

Normalising and superimposing it

listing 10.24–10.25 — display PYTHON

```python
heatmap = np.maximum(heatmap, 0)
heatmap /= np.max(heatmap)

heatmap = np.uint8(255 * heatmap)
jet_colors = cm.get_cmap("jet")(np.arange(256))[:, :3]
jet_heatmap = keras.utils.array_to_img(jet_colors[heatmap])
jet_heatmap = jet_heatmap.resize((img.shape[1], img.shape[0]))
jet_heatmap = keras.utils.img_to_array(jet_heatmap)

superimposed_img = jet_heatmap * 0.4 + img
keras.utils.array_to_img(superimposed_img).save("elephant_cam.jpg")
```

The result is the picture with the evidence highlighted — the artefact you would actually show a domain expert.

The two questions it answers

Why did it say African elephant?

Because **these pixels** carried the evidence. If they are the wrong pixels, you have found a spurious correlation.

Where is the elephant?

The heatmap localises the object without ever having been trained to do detection.

The first question is the one that matters in review: a model that is right **for the wrong reasons** passes every accuracy check and **fails the first time the background changes**

04

Visualising the latent space

Which inputs does the model consider similar?

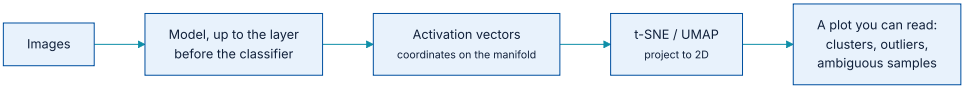
SECTION 10.4

Every model embeds its inputs in a latent space

This is chapter 5's manifold hypothesis, now made visible.

SECTION 10.4

How to look at it



Take the activations of one layer as coordinates, then project them down to two dimensions.

Any layer can be used — **each encodes a different latent space**, and they become progressively more semantically organised with depth. The usual choice is the layer just before the classifier.

SECTION 10.4

What it is actually good for

Finding outliers

Points sitting far from every cluster are usually **mislabelled or corrupt** — the noisy data of chapter 5.

Finding ambiguity

Points between two clusters are the samples the model finds **semantically ambiguous** — chapter 5's ambiguous features.

Improving the dataset

Both findings feed straight back into **cleaning the training set and refining the evaluation set**.

Which makes this the technique that closes the loop: it is an interpretation tool whose output is **a concrete list of samples to go and look at**

SECTION 10.4

Projecting it, in practice

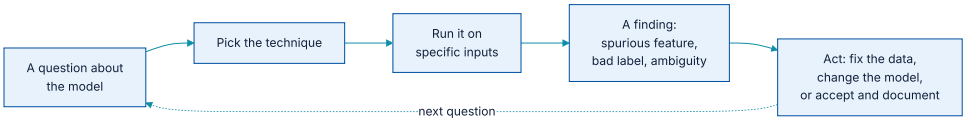
recording and projecting activations — PYTHON

```python
embedding_model = keras.Model(inputs=model.input,
                              outputs=model.layers[-2].output)   # before the classifier
embeddings = embedding_model.predict(dataset)

from sklearn.manifold import TSNE
projected = TSNE(n_components=2, init="pca").fit_transform(embeddings)

plt.scatter(projected[:, 0], projected[:, 1], c=labels, s=4, cmap="coolwarm")
```

Colour the points by their **label**, not by the prediction. A point whose colour disagrees with its neighbourhood is **either mislabelled or genuinely hard** — and both are worth knowing.

PRACTICE

Interpretation is a loop, not a report



The output of every technique here is meant to change something.

A heatmap that nobody acts on is decoration. The value is in the third and fourth boxes: **a finding, and a decision** — even when the decision is to accept the behaviour and write it down.

PRACTICE

And what these techniques do not give you

- They explain **this model on this input**. They are not a general statement about the model's behaviour on data it has not seen.
- A convincing heatmap is **not proof of a correct decision** — a model can look at the right region and still be wrong.
- None of them turn the model into a set of rules you can audit line by line. They make it **inspectable, not transparent**.

Which is worth saying plainly when a stakeholder asks whether the model can be "explained": these techniques answer a narrower question than the one usually being asked.

SUMMARY

Which technique answers which question

QUESTION YOU ARE ASKING	TECHNIQUE
What is each layer doing to my input?	**Intermediate activations** (§10.1)
What pattern does this filter respond to?	**Gradient ascent in input space** (§10.2)
Which part of the image drove this decision?	**Grad-CAM heatmap** (§10.3)
Which samples does my model confuse, and which are junk?	**Latent-space projection** (§10.4)

SUMMARY

What has to survive this chapter

1. **ConvNets are not black boxes.** Their representations are visual, and therefore visualisable.
2. **A deep network is an information distillation pipeline**: less about appearance, more about class, with rising sparsity at depth.
3. **Gradient ascent trains the image, not the weights** — same machinery, opposite target.
4. **Grad-CAM is a weighted average**, and it answers *why here?* as well as *where?*
5. **Latent-space projection finds the samples to go and inspect** — outliers and ambiguous cases.

NOTEBOOK

03_grad_cam.ipynb

NEXT

Chapter 11 — Image segmentation